

SBI 811: Computer Programming in Bioinformatics

Lecture 3 - Python Foundations

Welcome to Python

This notebook assumes that you have **never programmed before**. New terms are introduced gradually, and every important idea is followed by a small example.

You are not expected to memorize the entire Python language. Your first goal is to learn how to:

1. read a short program;
2. predict what it will do;
3. run it;
4. compare the result with your prediction; and
5. make a small, deliberate change.

Run the code cells in order from the beginning. Later cells use variables created in earlier cells. If the notebook behaves unexpectedly, select **Kernel** → **Restart Kernel and Run All Cells**.

Part 1 — What is programming?

1.1 A program is a set of instructions

A computer program is a sequence of instructions that a computer can execute.

Consider a simple biological task:

1. receive a DNA sequence;
2. determine its length;
3. count the number of G and C nucleotides;
4. calculate the percentage of the sequence made of G and C; and
5. display the result.

A human can follow these instructions. To make a computer follow them, we must express them precisely in a programming language.

1.2 What is Python?

Python is a programming language designed to be readable. It is widely used in bioinformatics, data analysis, automation, web development and scientific computing.

Python is **case-sensitive**. These names are different:

sequence
Sequence
SEQUENCE

1.3 Programming is not the same as typing commands blindly

Learning programming involves three connected skills:

- **Reading:** understanding what existing code does.
- **Writing:** expressing a task as correct instructions.
- **Debugging:** finding and correcting mistakes.

Mistakes are normal. An error message is information about where Python became unable to continue. Experienced programmers also encounter errors; they have simply learned how to read them calmly.

Part 2 — Understanding Jupyter Notebook

A Jupyter notebook combines:

- formatted explanatory text;
- executable code;
- results produced by the code; and
- figures or tables.

This combination is useful in science because an analysis can include both the reasoning and the computation.

2.1 Markdown cells

This explanation is inside a **Markdown cell**. Markdown cells contain headings, paragraphs, lists, links and explanations. They do not execute Python instructions.

2.2 Code cells

A **code cell** contains Python code. When you run the cell, the Python kernel executes its instructions.

2.3 The kernel

The **kernel** is the running Python process behind the notebook. It remembers variables created in executed cells.

2.4 Running a code cell

To run a selected cell:

- press **Shift + Enter**; or
- select the notebook's **Run** button.

The first code cell performs a calculation. Before running it, predict the answer.

```
In [ ]: 2 + 3
```

The value displayed below the cell is its output. In a notebook, the value of the final expression is normally displayed automatically.

Try changing one number, for example from `3` to `8`, and run the cell again. Programming becomes easier when you experiment with one small change at a time.

2.5 Execution order matters

Jupyter displays cells from top to bottom, but the kernel remembers the order in which cells were **executed**, not merely their visual position.

The notation beside a code cell may look like:

```
In [1]:
```

```
In [2]:
```

These numbers show execution order. If you run cell 10 before cell 4, the kernel executes cell 10 first.

For reproducible work, write notebooks that run correctly from the first cell to the last cell. Before submitting a notebook, restart the kernel and run all cells in order.

Part 3 — Your first Python output

3.1 The `print()` function

`print()` tells Python to display information.

In the example below:

- `print` is the function name;
- the parentheses contain the information given to the function; and
- the quotation marks identify text.

```
In [ ]: print("Welcome to SBI 811")
```

The quotation marks are not printed. They tell Python where the text begins and ends.

Python accepts single or double quotation marks for ordinary strings:

```
In [ ]: print("DNA sequence analysis")
print('DNA sequence analysis')
```

Both lines display the same text. Choose one style and use it consistently.

3.2 Printing several items

Separate items with commas inside `print()`. Python places a space between them.

```
In [ ]: print("Organism:", "Escherichia coli")
print("Sequence length:", 24)
```

Notice that `"Sequence length:"` is text, while `24` is a number. Python can print both in one statement.

Guided practice 1

Modify the cell below so that it displays:

```
Student: Your Name
Course: SBI 811
Topic: Python Foundations
```

```
In [ ]: # Replace the example information with your own details.
print("Student:", "Your Name")
print("Course:", "SBI 811")
print("Topic:", "Python Foundations")
```

Part 4 — Comments

A comment is an explanation written for human readers. Python ignores everything after `#` on that line.

```
In [ ]: # This line is a comment. Python does not execute it.
print("ATGCGT") # Display a short DNA sequence
```

Useful comments explain **why** something is done or clarify something that is not obvious.

Weak comment

```
length = 12 # Set length to 12
```

The code already says that.

More useful comment

```
length = 12 # Number of nucleotides after removing the FASTA header
```

Do not place a comment after every simple statement. Too many obvious comments make code harder to read.

Part 5 — Values, variables and assignment

5.1 What is a value?

A value is a piece of information used by a program. Examples include:

- the text `"ATGCGT"` ;
- the whole number `6` ;
- the decimal number `50.0` ; and
- the logical value `True` .

5.2 What is a variable?

A variable is a name that refers to a value.

```
sequence = "ATGCGT"
```

Read this statement as:

Assign the text `ATGCGT` to the variable named `sequence` .

The equals sign in this context means **assignment**, not a mathematical equation.

```
In [ ]: sequence = "ATGCGT"
        print(sequence)
```

Python stores the string and associates it with the name `sequence` . When `print(sequence)` runs, Python obtains the current value associated with that name.

5.3 Variables can change

```
In [ ]: sample_name = "sample_A"
        print(sample_name)

        sample_name = "sample_B"
        print(sample_name)
```

The second assignment replaces the previous value associated with `sample_name` . This is why it is called a variable: the referenced value can vary.

The old text has not been renamed; the variable now refers to a different value.

5.4 Choosing variable names

Good variable names make code easier to understand.

Avoid	Prefer
x	sequence_length
s	dna_sequence
n	sample_count
a1	gc_percentage

Rules

- A name may contain letters, digits and underscores.
- A name cannot begin with a digit.
- A name cannot contain spaces.
- A name cannot contain a hyphen.
- Python keywords such as `for`, `if` and `class` cannot be variable names.
- Names are case-sensitive.

Valid names

```
sequence
sequence_1
sample_name
gc_percentage
```

Invalid or problematic names

1sequence	begins with a digit
sample name	contains a space
gc-content	contains a hyphen
print	hides a useful built-in function

Guided practice 2 — Predict before running

What will this cell display? Write your prediction first.

```
In [ ]: organism = "Rhizobium"
gene_count = 6200

print("Organism:", organism)
print("Approximate genes:", gene_count)
```

Guided practice 3 — Correct the variable names

The following proposed names are unsuitable. Write a better name for each:

1. `1sample`
2. `dna sequence`
3. `gc-percent`
4. `x` when the value is the number of nucleotides

► Possible answers

Part 6 — Fundamental data types

A data type tells Python what kind of value it is working with and which operations are sensible.

We will begin with four types:

Python type	Meaning	Example
<code>str</code>	Text or a sequence of characters	<code>"ATGCGT"</code>
<code>int</code>	Whole number	<code>24</code>
<code>float</code>	Number containing a decimal point	<code>52.5</code>
<code>bool</code>	Logical value	<code>True</code> or <code>False</code>

The names are abbreviations:

- `str` means string;
- `int` means integer;
- `float` refers to a floating-point number; and
- `bool` means Boolean.

```
In [ ]: dna_sequence = "ATGCGT"
sequence_length = 6
gc_percentage = 50.0
passed_quality_check = True

print(dna_sequence)
print(sequence_length)
print(gc_percentage)
print(passed_quality_check)
```

6.1 Inspecting types with `type()`

The built-in `type()` function reports the type of a value.

```
In [ ]: print(type(dna_sequence))
print(type(sequence_length))
print(type(gc_percentage))
print(type(passed_quality_check))
```

Expected forms of output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

For now, focus on the final names: `str`, `int`, `float` and `bool`. The word `class` will be discussed much later.

6.2 Quotation marks change the type

Compare `24` with `"24"` :

```
In [ ]: number_value = 24
        text_value = "24"

        print(number_value, type(number_value))
        print(text_value, type(text_value))
```

Although they look similar when printed, they have different types:

- `24` is an integer and can be used directly in arithmetic.
- `"24"` is a string containing the characters `2` and `4`.

This distinction explains many beginner errors.

Guided practice 4 — Identify each type

Predict the type before running the cell.

```
In [ ]: sample_id = "SRR22587281"
        replicate_number = 3
        average_coverage = 27.6
        is_paired_end = True

        print(type(sample_id))
        print(type(replicate_number))
        print(type(average_coverage))
        print(type(is_paired_end))
```

Part 7 — Working with numbers

7.1 Arithmetic operators

Operator	Meaning	Example
<code>+</code>	addition	<code>5 + 2</code> gives <code>7</code>
<code>-</code>	subtraction	<code>5 - 2</code> gives <code>3</code>
<code>*</code>	multiplication	<code>5 * 2</code> gives <code>10</code>
<code>/</code>	division	<code>5 / 2</code> gives <code>2.5</code>
<code>//</code>	floor division	<code>5 // 2</code> gives <code>2</code>
<code>%</code>	remainder	<code>5 % 2</code> gives <code>1</code>
<code>**</code>	exponentiation	<code>5 ** 2</code> gives <code>25</code>

```
In [ ]: print("Addition:", 5 + 2)
        print("Subtraction:", 5 - 2)
        print("Multiplication:", 5 * 2)
        print("Division:", 5 / 2)
```



```
print("Floor division:", 5 // 2)
print("Remainder:", 5 % 2)
print("Power:", 5 ** 2)
```

/ and // are different

- / returns the ordinary division result.
- // removes the fractional part by rounding down.

The remainder operator % is useful for checking whether a number is divisible by another number. For example, a DNA sequence length is divisible by three when `length % 3` equals zero. We will use that idea later when studying conditions.

7.2 Order of operations

Python follows the usual arithmetic order:

1. parentheses;
2. powers;
3. multiplication and division;
4. addition and subtraction.

```
In [ ]: result_without_parentheses = 2 + 3 * 4
        result_with_parentheses = (2 + 3) * 4

        print(result_without_parentheses)
        print(result_with_parentheses)
```

Use parentheses when they make the calculation clearer, even if Python would produce the same result without them.

7.3 Arithmetic with variables

```
In [ ]: adenine_count = 8
        thymine_count = 7
        guanine_count = 5
        cytosine_count = 4

        sequence_length = (
            adenine_count
            + thymine_count
            + guanine_count
            + cytosine_count
        )

        print("Sequence length:", sequence_length)
```

The parentheses allow one logical calculation to continue across several lines. This formatting is easier to read than one very long line.

Guided practice 5 — Coverage calculation

Coverage can be simplified as:

$$\text{coverage} = \text{total sequenced bases} / \text{genome size}$$

Complete the cell to calculate coverage.

```
In [ ]: total_sequenced_bases = 230_000_000
genome_size = 4_600_000

coverage = total_sequenced_bases / genome_size

print("Approximate coverage:", coverage)
```

Underscores inside a number improve readability. Python interprets `230_000_000` as the same integer as `230000000`.

Part 8 — Working with strings

A string is a sequence of characters. DNA, RNA and protein sequences can be represented as strings.

```
dna = "ATGCGT"
protein = "MKWVTF"
sample_id = "sample_A"
```

The biological interpretation differs, but Python initially treats all three values as text.

```
In [ ]: dna = "ATGCGT"
protein = "MKWVTF"
sample_id = "sample_A"

print(dna)
print(protein)
print(sample_id)
```

8.1 String length with `len()`

`len()` reports the number of characters in a string.

```
In [ ]: dna = "ATGCGT"
sequence_length = len(dna)

print("Sequence:", dna)
print("Length:", sequence_length)
```

For this simple unbroken DNA string, the number of characters equals the number of nucleotides.

Spaces are also characters:

```
In [ ]: print(len("ATGCGT"))
print(len("ATG CGT"))
```

The second string is longer because the space is counted. This is one reason biological sequence data must be cleaned carefully.

8.2 Joining strings with `+`

When used with strings, `+` performs **concatenation**, meaning it joins strings.

```
In [ ]: first_part = "ATGC"
        second_part = "GTAA"
        complete_sequence = first_part + second_part

        print(complete_sequence)
```

Python does not insert a space automatically during concatenation.

8.3 Repeating a string with `*`

```
In [ ]: motif = "AT"
        repeated_motif = motif * 4

        print(repeated_motif)
```

The multiplication operator has different meanings for different types. With numbers it multiplies; with a string and integer it repeats the string.

8.4 Uppercase and lowercase

Python considers lowercase and uppercase characters different.

`atgc` is not the same string as `ATGC`

String methods can create standardized versions.

```
In [ ]: raw_sequence = "atGcgtAa"
        upper_sequence = raw_sequence.upper()
        lower_sequence = raw_sequence.lower()

        print("Original:", raw_sequence)
        print("Uppercase:", upper_sequence)
        print("Lowercase:", lower_sequence)
```

The dot in `raw_sequence.upper()` means: use the `upper` method belonging to this string.

Strings are **immutable**: `upper()` creates a new string; it does not silently alter the original string.

8.5 Removing surrounding whitespace

The `strip()` method removes whitespace from the beginning and end of a string.

```
In [ ]: raw_sequence = "  ATGCGTAA\n"
        clean_sequence = raw_sequence.strip()

        print("Raw representation:", repr(raw_sequence))
        print("Clean representation:", repr(clean_sequence))
```

`repr()` displays otherwise invisible characters clearly. `\n` represents a newline.

`strip()` does not remove spaces in the middle of a string. It removes surrounding whitespace only.

8.6 Counting characters

The string method `count()` counts non-overlapping occurrences.

```
In [ ]: dna = "ATGCGTAA"

        print("A count:", dna.count("A"))
        print("T count:", dna.count("T"))
        print("G count:", dna.count("G"))
        print("C count:", dna.count("C"))
```

The search is case-sensitive. If the sequence might contain lowercase letters, standardize it first:

```
dna = dna.upper()
```

8.7 Replacing characters

For a simple transcription example, replace `T` with `U`.

```
In [ ]: dna = "ATGCGTAA"
        rna = dna.replace("T", "U")

        print("DNA:", dna)
        print("RNA:", rna)
```

This is only a simple string transformation. Biological transcription involves a specific strand orientation and other considerations that will be discussed when the relevant biology is introduced.

Guided practice 6 — Clean and summarize a sequence

Complete the tasks using the supplied string:

1. remove surrounding whitespace;
2. convert the sequence to uppercase;
3. print the cleaned sequence;
4. print its length; and
5. print the number of `N` characters.

```
In [ ]: raw_sequence = " atgcnnactg \n"

clean_sequence = raw_sequence.strip().upper()

print("Clean sequence:", clean_sequence)
print("Length:", len(clean_sequence))
print("Unknown bases:", clean_sequence.count("N"))
```

Why do two methods appear together?

```
raw_sequence.strip().upper()
```

Python first creates the stripped string. It then converts that result to uppercase. This is called **method chaining**. Use it only when the order remains easy to understand.

Part 9 — Indexing: obtaining one character

Python numbers positions in a string beginning with **zero**.

For the string `ATGCGT` :

Character:	A	T	G	C	G	T
Index:	0	1	2	3	4	5

This is called zero-based indexing.

```
In [ ]: dna = "ATGCGT"

print("First nucleotide:", dna[0])
print("Second nucleotide:", dna[1])
print("Sixth nucleotide:", dna[5])
```

The number inside square brackets is the index.

Negative indexing

Negative indices count from the end:

Character:	A	T	G	C	G	T
Positive index:	0	1	2	3	4	5
Negative index:	-6	-5	-4	-3	-2	-1

```
In [ ]: dna = "ATGCGT"

print("Last nucleotide:", dna[-1])
print("Second-last nucleotide:", dna[-2])
```

A common indexing mistake

The first character is `dna[0]` , not `dna[1]` . This difference between human counting and Python indexing causes many beginner errors.

Guided practice 7 — Sequence positions

For `sequence = "GATTACA"`, predict and then print:

1. the first nucleotide;
2. the third nucleotide;
3. the final nucleotide; and
4. the second-last nucleotide.

```
In [ ]: sequence = "GATTACA"

print("First:", sequence[0])
print("Third:", sequence[2])
print("Final:", sequence[-1])
print("Second-last:", sequence[-2])
```

Part 10 — Slicing: obtaining several characters

A slice extracts part of a string.

`sequence[start:stop]`

The start position is included. The stop position is excluded.

This can be remembered as:

Start here, but stop before there.

```
In [ ]: dna = "ATGCGTAA"

print(dna[0:3])
print(dna[3:6])
```

For `dna[0:3]`, Python returns positions 0, 1 and 2. Position 3 is not included.

10.1 Omitting the start or stop

```
In [ ]: dna = "ATGCGTAA"

print("First three:", dna[:3])
print("From position 3 onward:", dna[3:])
print("Last three:", dna[-3:])
```

10.2 Slice step

A third value controls the step:

`sequence[start:stop:step]`

A step of 2 selects every second character.

```
In [ ]: dna = "ATGCGTAA"
```

```
print("Every second character:", dna[::2])
print("Reversed string:", dna[::-1])
```

`dna[::-1]` reverses the order of characters. It does **not** calculate a biological reverse complement because A has not been exchanged with T and G has not been exchanged with C.

Guided practice 8 — Extract a start codon

The first three characters of this sequence form a codon. Extract them using a slice.

```
In [ ]: sequence = "ATGGCCATTGTA"
first_codon = sequence[:3]

print("First codon:", first_codon)
```

Part 11 — Converting between types

Sometimes information is stored as text but needs to be used as a number.

Python provides conversion functions:

- `int()` creates an integer;
- `float()` creates a floating-point number; and
- `str()` creates a string.

```
In [ ]: read_count_text = "1250"
read_count = int(read_count_text)

print(read_count_text, type(read_count_text))
print(read_count, type(read_count))
print("Read count plus 50:", read_count + 50)
```

11.1 Why conversion is necessary

With strings, `+` joins text. With numbers, it performs addition.

```
In [ ]: print("12" + "3")
print(12 + 3)
```

The first result is `123` because two strings were joined. The second result is `15` because two integers were added.

Part 12 — Clear output with f-strings

An f-string places variable values inside text. Put `f` before the opening quotation mark and place variable names inside braces.

```
In [ ]: sample = "sample_A"
length = 24

print(f"The length of {sample} is {length} nucleotides.")
```

F-strings often produce clearer output than long comma-separated print statements.

12.1 Formatting decimal places

```
In [ ]: gc_percentage = 53.846153846

print(f"GC percentage: {gc_percentage:.2f}%")
```

Inside `{gc_percentage:.2f}`:

- `gc_percentage` is the value;
- `:` begins the formatting instruction;
- `.2f` means display a floating-point number with two digits after the decimal point.

Formatting changes how the result is displayed, not the underlying stored value.

Part 13 — Biological application: GC percentage

GC percentage describes the proportion of nucleotides that are G or C.

For a sequence of length L :

$$\text{GC percentage} = \frac{\text{number of G} + \text{number of C}}{L} \times 100$$

In plain language:

1. count G;
2. count C;
3. add the two counts;
4. divide by the total sequence length; and
5. multiply by 100.

```
In [ ]: dna = "ATGCGTAA"

sequence_length = len(dna)
g_count = dna.count("G")
c_count = dna.count("C")
gc_count = g_count + c_count
gc_percentage = (gc_count / sequence_length) * 100

print("Sequence:", dna)
print("Length:", sequence_length)
print("G count:", g_count)
print("C count:", c_count)
print(f"GC percentage: {gc_percentage:.2f}%")
```


13.1 Trace the values

For `ATGCGTAA` :

Variable	Value	Meaning
<code>sequence_length</code>	8	total characters/nucleotides
<code>g_count</code>	2	number of G characters
<code>c_count</code>	1	number of C characters
<code>gc_count</code>	3	G plus C
<code>gc_percentage</code>	37.5	$3/8 \times 100$

Tracing values step by step is a valuable debugging technique.

13.2 Clean before calculating

Real copied sequences may contain lowercase letters or surrounding whitespace. Standardize them before counting.

```
In [ ]: raw_dna = " atGCGtaa\n"
dna = raw_dna.strip().upper()

sequence_length = len(dna)
gc_count = dna.count("G") + dna.count("C")
gc_percentage = (gc_count / sequence_length) * 100

print("Clean sequence:", dna)
print("Length:", sequence_length)
print(f"GC percentage: {gc_percentage:.2f}%")
```

Guided practice 9 — Analyse another sequence

Use the sequence below to calculate:

1. sequence length;
2. A count;
3. T count;
4. G count;
5. C count; and
6. GC percentage to two decimal places.

```
In [ ]: dna = "AACCGGTTAAGCGC"

sequence_length = len(dna)
a_count = dna.count("A")
t_count = dna.count("T")
g_count = dna.count("G")
c_count = dna.count("C")
gc_percentage = ((g_count + c_count) / sequence_length) * 100

print("Sequence:", dna)
```

```
print("Length:", sequence_length)
print("A count:", a_count)
print("T count:", t_count)
print("G count:", g_count)
print("C count:", c_count)
print(f"GC percentage: {gc_percentage:.2f}%")
```

13.3 Check whether the counts are consistent

The four nucleotide counts should add up to the sequence length when the sequence contains only A, T, G and C.

```
In [ ]: counted_bases = a_count + t_count + g_count + c_count

print("Length:", sequence_length)
print("A + T + G + C:", counted_bases)
print("Counts match length:", counted_bases == sequence_length)
```

`==` compares two values and produces `True` or `False`. It is different from assignment:

- `=` assigns a value;
- `==` asks whether two values are equal.

We will use comparisons more extensively when learning conditions.

Part 14 — Scripts versus notebook cells

A Python script is a text file whose name normally ends in `.py`.

Example filename:

`sequence_summary.py`

A notebook is ideal for explanation and exploration. A script is useful when instructions should run together as one program.

14.1 Example script

```
# sequence_summary.py

dna = "ATGCGTAA"
length = len(dna)
gc_count = dna.count("G") + dna.count("C")
gc_percentage = (gc_count / length) * 100

print(f"Sequence: {dna}")
print(f"Length: {length}")
print(f"GC percentage: {gc_percentage:.2f}%")
```

Save it and run it from Ubuntu:

```
python sequence_summary.py
```

The `python` command starts Python and tells it to execute the file.

14.2 Create the script from this notebook

The following cell writes a small script into the current notebook directory. Run it only if you want the file to be created.

```
In [ ]: script_text = '''# sequence_summary.py

dna = "ATGCGTAA"
length = len(dna)
gc_count = dna.count("G") + dna.count("C")
gc_percentage = (gc_count / length) * 100

print(f"Sequence: {dna}")
print(f"Length: {length}")
print(f"GC percentage: {gc_percentage:.2f}%")
'''

with open("sequence_summary.py", "w") as script_file:
    script_file.write(script_text)

print("Created sequence_summary.py")
```

The file-writing syntax is provided here so that the notebook can produce a script. Detailed file handling will be taught later; you are not expected to memorize `with open(...)` in this lecture.

Part 15 — Reading errors without panic

An error message is also called a **traceback**. Read it from the bottom upward:

1. the final line names the error type and gives a short explanation;
2. the lines above show where it occurred; and
3. an arrow may point to the failing line.

The examples below catch errors safely so that the notebook continues running.

15.1 NameError

A `NameError` usually means that Python does not recognize a variable name.

```
In [ ]: try:
        print(dna_sequence) # Deliberate spelling mistake
except NameError as error:
    print(type(error).__name__ + ":", error)
```

Possible cause: `dna_sequence` was typed as `dna_sequunce`.

Checklist:

- Is the spelling identical?
- Is capitalization identical?

- Was the cell that creates the variable executed?

15.2 SyntaxError

A `SyntaxError` means that the code does not follow Python's grammatical rules.

Example invalid code:

```
print("ATGCGT"
```

The closing parenthesis is missing.

Correct code:

```
print("ATGCGT")
```

Other common causes include missing quotation marks and misplaced punctuation.

15.3 TypeError

A `TypeError` can occur when an operation is not valid for the supplied types.

```
In [ ]: try:
        result = "Sequence length: " + 24
    except TypeError as error:
        print(type(error).__name__ + ":", error)
```

Python cannot directly concatenate a string and an integer with `+`.

Correct alternatives:

```
print("Sequence length:", 24)
print("Sequence length: " + str(24))
print(f"Sequence length: {24}")
```

```
In [ ]: print("Sequence length:", 24)
        print("Sequence length: " + str(24))
        print(f"Sequence length: {24}")
```

15.4 IndexError

An `IndexError` occurs when the requested position does not exist.

```
In [ ]: short_sequence = "ATGC"

        try:
            print(short_sequence[10])
        except IndexError as error:
            print(type(error).__name__ + ":", error)
```

The string has valid positive indices 0–3. Index 10 lies outside the string.

Use `len(short_sequence)` to check its size.

15.5 ZeroDivisionError

GC percentage requires division by sequence length. An empty sequence has length zero.

```
In [ ]: empty_sequence = ""

try:
    gc_percentage = (
        (empty_sequence.count("G") + empty_sequence.count("C"))
        / len(empty_sequence)
        * 100
    )
except ZeroDivisionError as error:
    print(type(error).__name__ + ":", error)
```

Later, conditions will allow a program to check for an empty sequence before performing the calculation.

Part 16 — Integrated guided activity

You have received this copied DNA sequence:

atggccaattggccnn

Write code that:

1. stores the raw text;
2. removes surrounding whitespace;
3. converts it to uppercase;
4. prints the cleaned sequence;
5. prints its length;
6. prints A, T, G, C and N counts;
7. calculates GC percentage using the full cleaned length; and
8. displays the percentage to two decimal places.

Try the blank cell before opening the solution.

```
In [ ]: # Write your solution here.
raw_sequence = " atggccaattggccnn "
```

► Open one possible solution after attempting the activity

Part 17 — AI-assisted challenge

Ask an AI assistant to explain the following code **line by line in beginner language**:

```
dna = "AACCGGTTA"
length = len(dna)
gc_count = dna.count("G") + dna.count("C")
gc_percent = (gc_count / length) * 100
print(f"GC: {gc_percent:.1f}%")
```

Then verify the explanation yourself:

1. What value is stored in `length` ?
2. What value is stored in `gc_count` ?
3. Why is the result multiplied by 100?
4. What does `.1f` change?
5. Change the sequence and calculate the expected result manually.

Responsible AI use

- Do not accept an explanation you cannot restate yourself.
- Run the code and compare the result with a manual calculation.
- Ask the AI to simplify unfamiliar terms.
- Never paste passwords, tokens, confidential data or unpublished sensitive sequences into a public AI system.

Part 18 — Independent practice

Activity A — Variables and types

Create variables for:

- sample identifier;
- organism name;
- sequence length;
- GC percentage; and
- whether the sample passed quality control.

Print each value and its type.

Activity B — Sequence inspection

For `GCTATGCCATGA` , print:

- its length;
- its first nucleotide;
- its last nucleotide;
- its first three nucleotides;
- its final three nucleotides; and
- the reversed string.

Activity C — Nucleotide summary

For `ACGTTGCAACGGTT` , calculate the count and percentage of each nucleotide.

Activity D — Small script

Create `my_sequence_summary.py`, place one DNA sequence inside it, calculate length and GC percentage, and run it from Ubuntu.

Part 19 — Final review

Complete these statements without looking back:

1. Python is __.
2. A variable is __.
3. A string is __.
4. An integer is __.
5. A float is _
6. `=` is used to _.
7. `==` is used to __.
8. `len(sequence)` returns _.
9. `sequence[0]` returns __.
10. `sequence[:3]` returns __.
11. `.upper()` __.
12. `.strip()` __.
13. `.count("G")` __.
14. A traceback is __.

If you can explain these ideas and calculate GC percentage without copying an answer, you have achieved the main goals of Lecture 3.

Compact reference sheet

```
# Display information
print("Hello")
```

```
# Variables
sequence = "ATGCGT"
length = len(sequence)
```

```
# Types
type(sequence)
type(length)
```

```
# String operations
sequence.upper()
sequence.lower()
sequence.strip()
sequence.count("G")
sequence.replace("T", "U")
```

```
# Indexing and slicing
sequence[0]      # first character
sequence[-1]     # last character
sequence[:3]     # first three characters
```

```
sequence[-3:]      # last three characters
sequence[::-1]     # reversed string

# Type conversion
int("24")
float("52.5")
str(24)

# Formatted output
print(f"Length: {length}")
print(f"GC: {gc_percentage:.2f}%")
```

Before submitting your work

- [] Restart the kernel and run all cells in order.
- [] Read every output and check that it is reasonable.
- [] Replace placeholder names with your details where required.
- [] Complete the guided and independent activities.
- [] Do not leave unexplained error messages in the notebook.
- [] Save the notebook with a meaningful filename.
- [] Commit and push it to your `SBI811-Exercises` repository when instructed.